Security Audit Report

Neutron Auction and Undelegations Contract, Chain Upgrade

Phase 2

Report 1

v1.0

April 9, 2025

Table of Contents

Table of Contents 2
License 3
Disclaimer 4
Introduction 5
Purpose of This Report 5
Codebase Submitted for the Audit 5
Methodology 6
Functionality Overview 6
How to Read This Report 7
Code Quality Criteria 8
Summary of Findings 9
Detailed Findings 10
1. tick_undelegation_amount accepts zero, causing undelegation messages with a zero coin amount 10
2. tick_period_seconds can be set too low, risking unbonding entry exhaustion and denial of undelegation 10
3. tick_undelegation_amount is applied per validator, contradicting its README description as a total cap 11
4. Overly permissive message dispatch enabling privilege escalation via arbitrary CosmosMsg execution 11
5. Multiple entry points emit insufficient event attributes 12
6. Direct ownership transfer in auth-proxy and undelegations-manager allows accidental loss of contract control 13
7. Error variants in ContractError are unused 13
8. Tick and Burn messages can be called by any address 14
9. Incomplete pagination handling leads to delegation starvation and state inconsistency 14

License

THIS WORK IS LICENSED UNDER A CREATIVE COMMONS ATTRIBUTION-NODERIVATIVES 4.0 INTERNATIONAL LICENSE.

Disclaimer

THE CONTENT OF THIS AUDIT REPORT IS PROVIDED “AS IS”, WITHOUT REPRESENTATIONS AND WARRANTIES OF ANY KIND.

THE AUTHOR AND HIS EMPLOYER DISCLAIM ANY LIABILITY FOR DAMAGE ARISING OUT OF, OR IN CONNECTION WITH, THIS AUDIT REPORT.

THIS AUDIT REPORT WAS PREPARED EXCLUSIVELY FOR AND IN THE INTEREST OF THE CLIENT AND SHALL NOT CONSTRUE ANY LEGAL RELATIONSHIP TOWARDS THIRD PARTIES. IN PARTICULAR, THE AUTHOR AND HIS EMPLOYER UNDERTAKE NO LIABILITY OR RESPONSIBILITY TOWARDS THIRD PARTIES AND PROVIDE NO WARRANTIES REGARDING THE FACTUAL ACCURACY OR COMPLETENESS OF THE AUDIT REPORT.

FOR THE AVOIDANCE OF DOUBT, NOTHING CONTAINED IN THIS AUDIT REPORT SHALL BE CONSTRUED TO IMPOSE ADDITIONAL OBLIGATIONS ON COMPANY, INCLUDING WITHOUT LIMITATION WARRANTIES OR LIABILITIES.

COPYRIGHT OF THIS REPORT REMAINS WITH THE AUTHOR.

This audit has been performed by

Oak Security GmbH

https://oaksecurity.io/
info@oaksecurity.io

Introduction

Purpose of This Report

Oak Security GmbH has been engaged by Hadron Labs to perform a security audit of Neutron Auction and Undelegations Contract, Chain Upgrade.

The objectives of the audit are as follows:

1. Determine the correct functioning of the protocol, in accordance with the project specification.
2. Determine possible vulnerabilities, which could be exploited by an attacker.
3. Determine smart contract bugs, which might lead to unexpected behavior.
4. Analyze whether best practices have been applied during development.
5. Make recommendations to improve code safety and readability.

This report represents a summary of the findings.

As with any code audit, there is a limit to which vulnerabilities can be found, and unexpected execution paths may still be possible. The author of this report does not guarantee complete coverage (see disclaimer).

Codebase Submitted for the Audit

The audit has been performed on the following target:

Repository	https://github.com/oak-security/audit-neutron-org-neutron-umc
Commit	`9f90223bc6dd7541efd067a4d46773e365c86165`
Scope	All contracts were in scope.
Fixes verified at commit	`76aac6f3a9746cf13d2b8e99b5f927fa692f0b8c` Note that only fixes to the issues described in this report have been reviewed at this commit. Any further changes such as additional features have not been reviewed.

Methodology

The audit has been performed in the following steps:

1. Gaining an understanding of the code base's intended purpose by reading the available documentation.
2. Automated source code and dependency analysis.
3. Manual line-by-line analysis of the source code for security vulnerabilities and use of best practice guidelines, including but not limited to:
    a. Race condition analysis
    b. Under-/overflow issues
    c. Key management vulnerabilities
4. Report preparation

Functionality Overview

The UMC contract manages configuration for an `owner` and a `delegator_contract`, enforces a per-tick undelegation cap and minimum tick interval, and exposes execution entry points for performing periodic undelegations and burning untrn by querying delegations/balances and forwarding structured messages to the delegator contract, along with owner-only configuration updates and a configuration query.

The auth-proxy contract maintains a single `owner` and provides an owner-only forwarding mechanism to emit arbitrary `CosmosMsg` messages from the proxy, plus an owner update entry point.

How to Read This Report

This report classifies the issues found into the following severity categories:

Severity	Description
Critical	A serious and exploitable vulnerability that can lead to loss of funds, unrecoverable locked funds, or catastrophic denial of service.
Major	A vulnerability or bug that can affect the correct functioning of the system, lead to incorrect states or denial of service.
Minor	A violation of common best practices or incorrect usage of primitives, which may not currently have a major impact on security, but may do so in the future or introduce inefficiencies.
Informational	Comments and recommendations of design decisions or potential optimizations, that are not relevant to security. Their application may improve aspects, such as user experience or readability, but is not strictly necessary. This category may also include opinionated recommendations that the project team might not share.

The status of an issue can be one of the following: **Pending**, **Acknowledged**, **Partially Resolved**, or **Resolved**.

Note that audits are an important step to improving the security of smart contracts and can find many issues. However, auditing complex codebases has its limits and a remaining risk is present (see disclaimer).

Users of the system should exercise caution. In order to help with the evaluation of the remaining risk, we provide a measure of the following key indicators: **code complexity**, **code readability**, **level of documentation**, and **test coverage**. We include a table with these criteria below.

Note that high complexity or low test coverage does not necessarily equate to a higher risk, although certain bugs are more easily detected in unit testing than in a security audit and vice versa.

Code Quality Criteria

The auditor team assesses the codebase's code quality criteria as follows:

Criteria	Status	Comment
Code complexity	Low	Logic is focused around a few clear flows (undelegation ticking, forwarding, config updates) with some state and rate-limiting, but no heavy algorithms or deeply nested control structures.
Code readability and clarity	Medium-High	-
Level of documentation	Medium	The README and scripts describe the overall workflow.
Test coverage	Low	Unit tests are not implemented, but an end-to-end test script is available.

Summary of Findings

No	Description	Severity	Status
1	`tick_undelegation_amount` accepts zero, causing undelegation messages with a zero coin amount	Minor	Resolved
2	`tick_period_seconds` can be set too low, risking unbonding entry exhaustion and denial of undelegation	Minor	Acknowledged
3	`tick_undelegation_amount` is applied per validator, contradicting its README description as a total cap	Minor	Resolved
4	Overly permissive message dispatch enabling privilege escalation via arbitrary `CosmosMsg` execution	Minor	Acknowledged
5	Multiple entry points emit insufficient event attributes	Informational	Resolved
6	Direct ownership transfer in auth-proxy and undelegations-manager allows accidental loss of contract control	Informational	Acknowledged
7	Error variants in `ContractError` are unused	Informational	Resolved
8	`Tick` and `Burn` messages can be called by any address	Informational	Resolved
9	Incomplete pagination handling leads to delegation starvation and state inconsistency	Informational	Acknowledged

Detailed Findings

1. `tick_undelegation_amount` accepts zero, causing undelegation messages with a zero coin amount

Severity: Minor

In `contracts/undelegations-manager/src/contract.rs:27-35` and `205-207`, neither `instantiate` nor `execute_update_config` validates that `tick_undelegation_amount` is greater than zero.

When `tick_undelegation_amount` is zero, the `execute_tick` loop in line `164` computes `amount.min(0)` for each delegation, producing a `StakingMsg::Undelegate` with a zero coin amount. The Cosmos SDK rejects zero-amount undelegations, causing every subsequent `Tick` call to fail for as long as the value remains zero.

Recommendation

We recommend adding a validation check that `tick_undelegation_amount` is strictly greater than zero in both `instantiate` and `execute_update_config`, returning an error otherwise. More generally, we recommend defining explicit lower and upper bounds for all numeric configuration parameters.

Status: Resolved

2. `tick_period_seconds` can be set too low, risking unbonding entry exhaustion and denial of undelegation

Severity: Minor

In `contracts/undelegations-manager/src/contract.rs:27-35` and `208-210`, `tick_period_seconds` is accepted without any lower bound. On Neutron, the staking module enforces a maximum of 7 concurrent unbonding entries per `(delegator, validator)` pair within the 20-day unbonding period.

If `tick_period_seconds` is set below approximately 3 days (20 days / 7 entries), the contract may exhaust the unbonding entry limit before existing entries mature. Once the limit is reached, any further `Tick` call that tries to undelegate from a validator with 7 active entries will fail, and the contract will be unable to process undelegations for those validators until entries mature.

Recommendation

We recommend enforcing a lower bound on `tick_period_seconds` of at least `unbonding_period / max_entries` (approximately 246,000 seconds, for current Neutron parameters) in both `instantiate` and `execute_update_config`.

Status: Acknowledged

3. `tick_undelegation_amount` is applied per validator, contradicting its README description as a total cap

Severity: Minor

In `contracts/undelegations-manager/src/contract.rs:164`, the undelegation amount per validator is capped with `amount.min(config.tick_undelegation_amount)`.

With N validators, the total amount undelegated in a single tick can reach up to `N × tick_undelegation_amount`. However, `README.md:9` describes this parameter as "Maximum amount of `untrn` to undelegate in a single tick from all delegated validators," implying a global cap across all validators combined.

This discrepancy may cause operators to underestimate the actual undelegation rate when configuring the contract.

Recommendation

We recommend either updating the README to clarify that `tick_undelegation_amount` is a per-validator limit, or adjusting the implementation to enforce a global cap across all validators in a single tick.

Status: Resolved

4. Overly permissive message dispatch enabling privilege escalation via arbitrary `CosmosMsg` execution

Severity: Minor

In `contracts/auth-proxy/src/contract.rs:34-48`, the `execute_forward` function implements an over-privileged dispatcher pattern by allowing the contract owner to forward arbitrary `CosmosMsg` messages without restriction. While access control ensures only the owner can invoke this function, the absence of message-type validation violates the principle of least privilege.

If the owner account is compromised, or if downstream contract assumptions evolve, a malicious actor could exploit this functionality to execute unintended operations such as arbitrary contract calls, token transfers, IBC interactions, or other chain-supported message types. This significantly increases the attack surface, potentially enabling unauthorized state transitions, fund movement, or cross-contract interactions.

Given that the intended use case only requires `StakingMsg::Undelegate` and `BankMsg::Burn`, accepting all `CosmosMsg` variants introduces unnecessary systemic risk and weakens the security model.

Recommendation

We recommend enforcing strict input validation by explicitly whitelisting permitted message types (e.g., `StakingMsg::Undelegate` and `BankMsg::Burn`) and rejecting all other variants.

Status: Acknowledged

5. Multiple entry points emit insufficient event attributes

Severity: Informational

Multiple entry points emit insufficient event attributes:

- In `contracts/undelegations-manager/src/contract.rs:38`, the `instantiate` entry point only emits `action` as a response attribute, omitting `owner`, `delegator_contract`, `tick_undelegation_amount`, and `tick_period_seconds`.
- In `contracts/undelegations-manager/src/contract.rs:180-183`, the `execute_tick` response emits `total_delegated` but not the total amount actually undelegated in the current tick.
- In `contracts/undelegations-manager/src/contract.rs:213`, the `UpdateConfig` handler returns only `action=update_config`, without any of the updated field values.

This makes it harder to verify or audit the contract state from transaction history alone.

Recommendation

We recommend emitting all relevant configuration fields as attributes in the `instantiate` response, accumulating and emitting the total undelegated amount in the `execute_tick` response, and emitting the new values of all updated fields in `execute_update_config`.

Status: Resolved

6. Direct ownership transfer in auth-proxy and undelegations-manager allows accidental loss of contract control

Severity: Informational

In `contracts/auth-proxy/src/contract.rs:50-65`, the `execute_update_owner` handler immediately replaces the stored owner with the new address provided by the caller.

Similarly, in `contracts/undelegations-manager/src/contract.rs:199-201`, `execute_update_config` replaces the `owner` field in a single step.

While this is common practice, it presents a risk for the ownership of the contract to become lost if the owner transfers ownership to an incorrect address. A two-step ownership transfer will allow the current owner to propose a new owner, and then the account that is proposed as the new owner may call a function that will allow them to claim ownership and actually execute the config update.

Recommendation

We recommend implementing a two-step ownership transfer. The flow can be as follows:

- The current owner proposes a new owner address that is validated and lowercased.
- The new owner account claims ownership, which applies the configuration changes.

Status: Acknowledged

7. Error variants in `ContractError` are unused

Severity: Informational

In `contracts/undelegations-manager/src/error.rs:12-29`, the `ContractError` enum declares six variants (`DepositPeriodEnded`, `DepositPeriodNotEnded`, `ZeroDeposit`, `AuctionSettled`, `NoClaimableAmount`, and `InsufficientFunds`) that are never constructed or returned anywhere in the contract. These appear to be remnants of a previous design. Dead variants add noise to the error type and may mislead integrators into expecting conditions that are never actually enforced.

Recommendation

We recommend removing all unused error variants from `ContractError`.

Status: Resolved

8. `Tick` and `Burn` messages can be called by any address

Severity: Informational

In `contracts/undelegations-manager/src/contract.rs:74` and `112`, neither `execute_burn` nor `execute_tick` restricts the caller. Any address can invoke either message at any time (subject to the tick cooldown in the case of `Tick`). The absence of access control means that any external actor can trigger undelegation or burn operations.

Recommendation

If permissionless execution is intentional for both messages, we recommend documenting this design decision explicitly. Otherwise, we recommend restricting each message to authorized callers, for example, limiting `Tick` to the cron address and `Burn` to the owner or the undelegations-manager contract itself.

Status: Resolved

9. Incomplete pagination handling leads to delegation starvation and state inconsistency

Severity: Informational

In `contracts/undelegations-manager/src/contract.rs:112-131`, the `execute_tick` function performs a delegation query using a hardcoded pagination limit (`limit: 100`) without iterating over additional pages. This results in incomplete processing when the number of delegations exceeds the specified limit.

A malicious actor or unfavorable validator ordering could exploit this limitation to ensure certain delegations are consistently excluded from processing, leading to starvation.

Recommendation

We recommend implementing a batching mechanism with explicit continuation state stored on-chain to guarantee fair processing across multiple ticks.

Status: Acknowledged